



JSON. JSONSERIALIZER

JSON

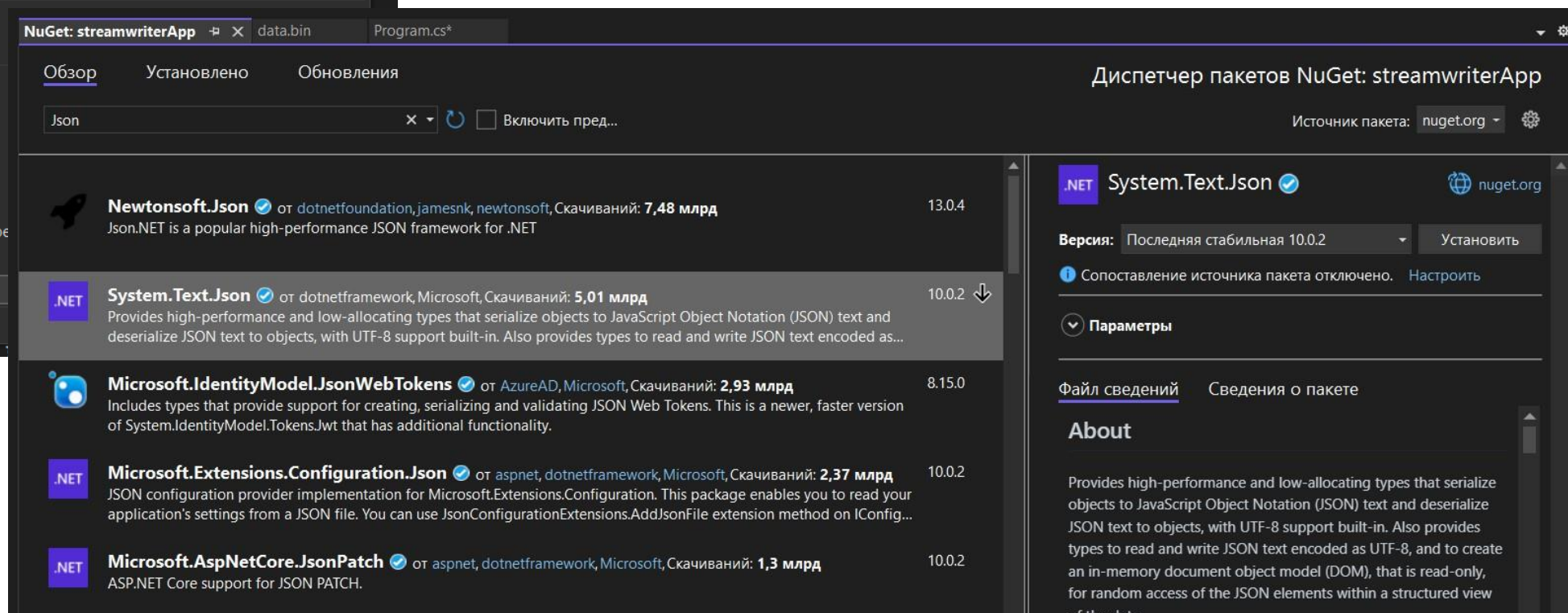
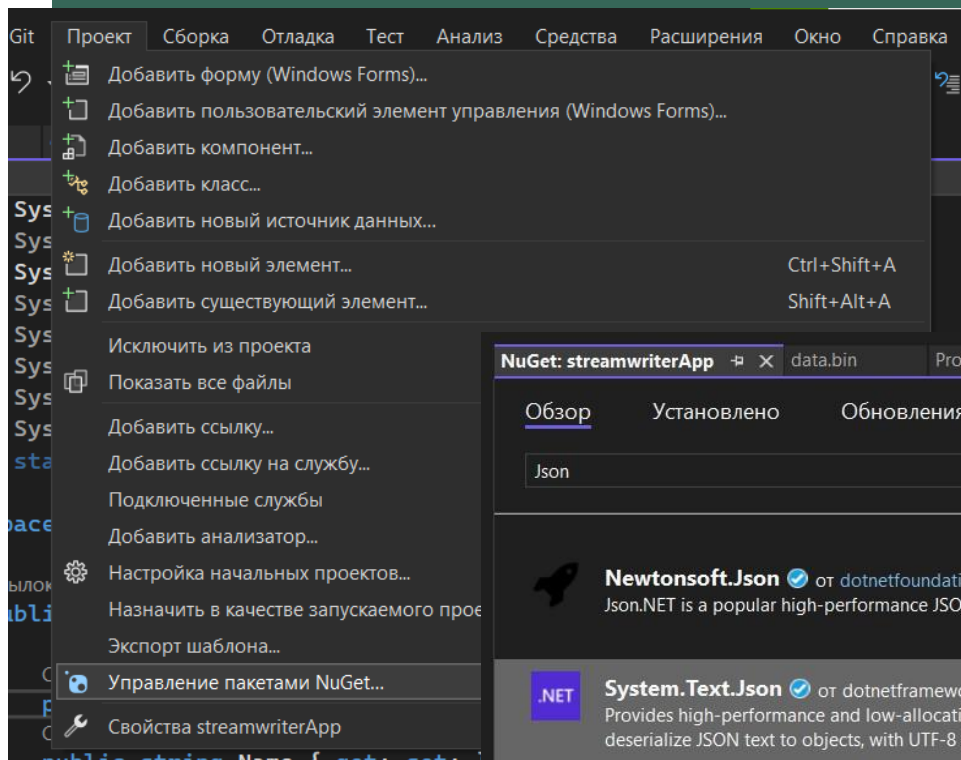
(JavaScript Object Notation) является одним из наиболее популярных форматов для хранения и передачи данных. И платформа .NET предоставляет функционал для работы с JSON. Основная функциональность по работе с JSON сосредоточена в пространстве имен **System.Text.Json**. Ключевым типом является класс **JsonSerializer**, который и позволяет сериализовать объект в json и, наоборот, десериализовать код json в объект C#.



JSON FILE

ЗАГРУЗКА ПАКЕТА

- Выбираем System.Text.Json и нажимаем установить, принимаем все соглашения. Установка может занять долгое время(до 5ти минут) и требует подключение к интернету, важно не закрывать вкладку NuGet или приложение.



СТРУКТУРА JSON

```
[  
  {  
    "Id": 1,  
    "Name": "Ivan Ivanov",  
    "Age": 20,  
    "Group": "CS-101"  
  },  
  {  
    "Id": 2,  
    "Name": "Maria Petrova",  
    "Age": 19,  
    "Group": "CS-101"  
  },  
  {  
    "Id": 3,  
    "Name": "Alex Sidorov",  
    "Age": 21,  
    "Group": "CS-102"  
  },  
  {  
    "Id": 4,  
    "Name": "Elena Kuznetsova",  
    "Age": 20,  
    "Group": "CS-102"  
  }  
]
```

Для работы с Json файлами, требуется преобразовывать объект C# в строку формата Json в это называется - **сериализация**. Чтобы работать с объектом, требуется преобразовывать строку формата Json в объект C# это называется - **десериализация**.

```
Ссылка: 3  
public class Student  
{  
    Ссылка: 4  
    public int Id { get; set; }  
    Ссылка: 4  
    public string Name { get; set; }  
    Ссылка: 4  
    public int Age { get; set; }  
    Ссылка: 4  
    public string Group { get; set; }  
}
```

ПРИМЕР СЕРИАЛИЗАЦИИ И ДЕСЕРИАЛИЗАЦИИ

```
Ссылка: 0
static void Main(string[] args)
{
    Person tom = new Person("Tom", 37);
    // сериализуем объект класса Person в строку вида Json
    string json = JsonSerializer.Serialize(tom);
    Console.WriteLine(json);
    // десериализуем строку вида Json в объект класса Person
    Person restoredPerson = JsonSerializer.Deserialize<Person>(json);
    Console.WriteLine(restoredPerson.Name); // Tom
}

Ссылка: 5
class Person
{
    Ссылка: 2
    public string Name { get; }
    Ссылка: 1
    public int Age { get; set; }
    Ссылка: 1
    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }
}
```

Рассмотрим применение класса на простом примере. Сериализуем и десериализуем простейший объект:

Консоль отладки Microsoft Visual Studio

```
{"Name": "Tom", "Age": 37}
Tom
```

МЕТОД SERIALIZE()

Для сохранения объекта в json в классе JsonSerializer определен статический метод **Serialize()** и его асинхронный двойник **SerializeAsync()**, которые имеют ряд перегруженных версий **для записи в файл**. Некоторые из них:

- **string Serialize(Object obj, Type type, JsonSerializerOptions options):**
сериализует объект obj типа type и возвращает код json в виде строки. Последний необязательный параметр options позволяет задать дополнительные опции сериализации
- **string Serialize<T>(Stream utf8Json, T value, JsonSerializerOptions? options = null):** типизированная версия сериализует объект value типа T и записывает код json в виде строки в объект **Stream**.
- **Task SerializeAsync(Stream utf8Json, Object obj, Type type, JsonSerializerOptions options):** сериализует объект obj типа type и записывает его в поток utf8Json. Последний необязательный параметр options позволяет задать дополнительные опции сериализации
- **Task SerializeAsync<T>(Stream utf8Json, T obj, JsonSerializerOptions options):** типизированная версия сериализует объект obj типа T в поток utf8Json.

МЕТОД SERIALIZE()

string Serialize(Stream str, Object obj, Type type, JsonSerializerOptions options): сериализует объект obj типа type и возвращает код json в виде строки. Последний необязательный параметр options позволяет задать дополнительные опции сериализации

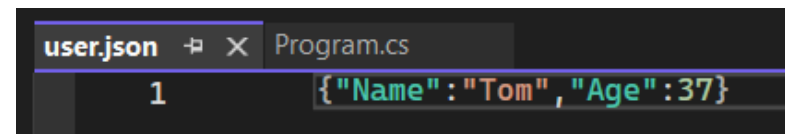
```
JsonSerializerOptions options = new JsonSerializerOptions()
{
    WriteIndented = true
};
using (FileStream fs = new FileStream("user.json", FileMode.Create))
{
    Person tom = new Person("Tom", 37);
    //JsonSerializer.Serialize<Person>(fs, tom, options);
    //JsonSerializer.Serialize(fs, tom);
    JsonSerializer.Serialize(fs, tom, typeof(Person));
    Console.WriteLine("Data has been saved to file");
}
```

```
class Person
{
    Ссылка: 2
    public string Name { get; }
    Ссылка: 2
    public int Age { get; set; }
    Ссылка: 1
    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }
}
```

МЕТОД SERIALIZE()

string Serialize<T>(Stream utf8Json, T value, JsonSerializerOptions? options = null):
типизированная версия сериализует объект **value** типа **T** и записывает код json в виде строки в объект **Stream**.

```
static void Main(string[] args)
{
    using (FileStream fs = new FileStream("user.json", FileMode.OpenOrCreate))
    {
        Person tom = new Person("Tom", 37);
        JsonSerializer.Serialize<Person>(fs, tom);
        Console.WriteLine("Data has been saved to file");
    }
}
```



The screenshot shows a code editor with two tabs: 'user.json' and 'Program.cs'. The 'user.json' tab is active, displaying the JSON string: `{"Name": "Tom", "Age": 37}`. The 'Program.cs' tab is also visible but empty.

```
Serialize<Person>(fs, tom);
```

```
line("D
```

 **void JsonSerializer.Serialize<Person>(Stream utf8Json, Person value, [JsonSerializerOptions? options = null])**
Converts the provided value to UTF-8 encoded JSON text and write it to the **Stream**.

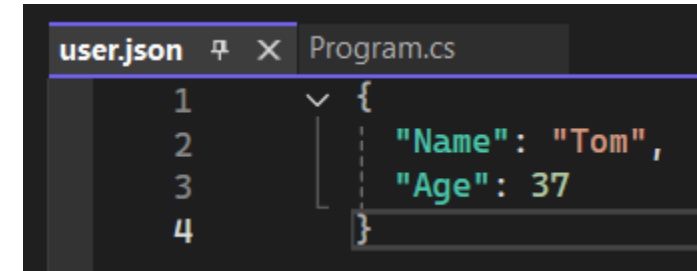
МЕТОД SERIALIZE() - JSONSERIALIZEROPTIONS

По умолчанию JsonSerializer сериализует объекты в минимифицированный код. С помощью дополнительного параметра типа **JsonSerializerOptions** можно настроить механизм сериализации/десериализации, используя свойства JsonSerializerOptions. Некоторые из его свойств:

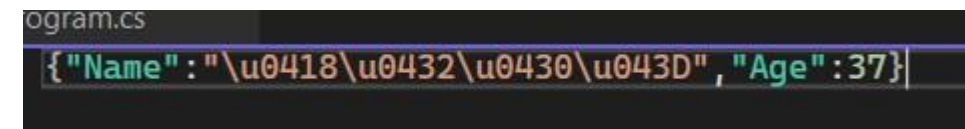
- **AllowTrailingCommas**: устанавливает, надо ли добавлять после последнего элемента в json запятую. Если равно true, запятая добавляется
- **DefaultIgnoreCondition**: устанавливает, будут ли сериализоваться/десериализоваться в json свойства со значениями по умолчанию
- **IgnoreReadOnlyProperties**: аналогично устанавливает, будут ли сериализоваться свойства, предназначенные только для чтения
- **WriteIndented**: устанавливает, будут ли добавляться в json пробелы (условно говоря, для красоты). Если равно true устанавливаются дополнительные пробелы
- **Encoder**: возвращает или задает кодировщик (объект JavaScriptEncoder), используемый при экранировании строк, или null для использования кодировщика по умолчанию.

МЕТОД SERIALIZE() - JSONSERIALIZEROPTIONS

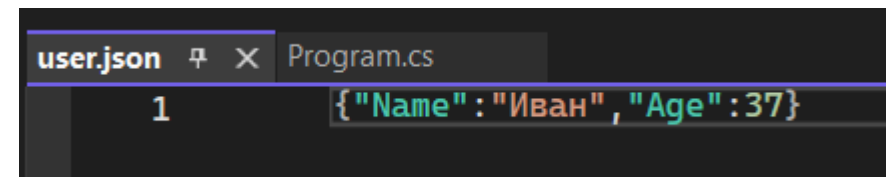
```
// создаём настройки для сериализации.  
// WriteIndented = true устанавливаются дополнительные пробелы  
// AllowTrailingCommas = true устанавливаются запятые  
JsonSerializerOptions options = new JsonSerializerOptions()  
{  
    WriteIndented = true,  
    AllowTrailingCommas = true  
};  
using (FileStream fs = new FileStream("user.json", FileMode.OpenOrCreate))  
{  
    Person tom = new Person("Tom", 37);  
    JsonSerializer.Serialize<Person>(fs, tom, options);  
    Console.WriteLine("Data has been saved to file");  
}
```



```
user.json  [icon] X Program.cs  
1 {  
2   "Name": "Tom",  
3   "Age": 37  
4 }
```



```
Program.cs  
1 {"Name": "\u0418\u0432\u0430\u043d", "Age": 37}  
2  
3  
4
```

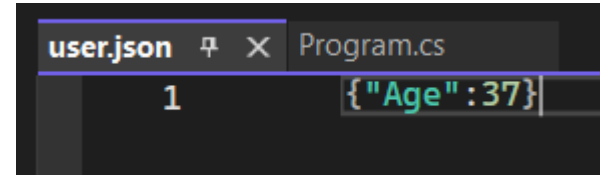


```
user.json  [icon] X Program.cs  
1 {"Name": "Иван", "Age": 37}  
2  
3  
4
```

```
JsonSerializerOptions options = new JsonSerializerOptions()  
{  
    Encoder = System.Text.Encodings.Web.JavaScriptEncoder.UnsafeRelaxedJsonEscaping  
};  
using (FileStream fs = new FileStream("user.json", FileMode.Create))  
{  
    Person tom = new Person("Иван", 37);  
    JsonSerializer.Serialize<Person>(fs, tom, options);  
    Console.WriteLine("Data has been saved to file");  
}
```

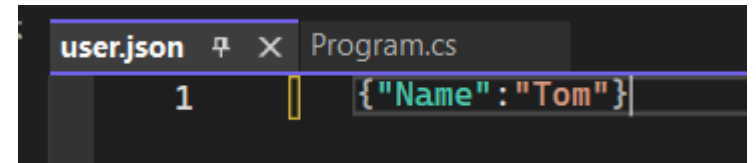
МЕТОД SERIALIZE() - JSONSERIALIZEROPTIONS

```
JsonSerializerOptions options = new JsonSerializerOptions()
{
    DefaultIgnoreCondition = System.Text.Json.Serialization.JsonIgnoreCondition.WhenWritingNull
};
using (FileStream fs = new FileStream("user.json", FileMode.Create))
{
    Person tom = new Person(null, 37);
    JsonSerializer.Serialize<Person>(fs, tom, options);
    Console.WriteLine("Data has been saved to file");
}
```



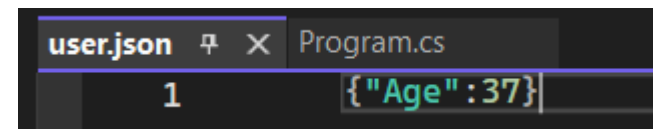
user.json 1 {"Age": 37}

```
JsonSerializerOptions options = new JsonSerializerOptions()
{
    DefaultIgnoreCondition = System.Text.Json.Serialization.JsonIgnoreCondition.WhenWritingDefault
};
using (FileStream fs = new FileStream("user.json", FileMode.Create))
{
    Person tom = new Person("Tom", 0);
    JsonSerializer.Serialize<Person>(fs, tom, options);
    Console.WriteLine("Data has been saved to file");
}
```



user.json 1 {"Name": "Tom"}

```
JsonSerializerOptions options = new JsonSerializerOptions()
{
    IgnoreReadOnlyProperties = true
};
using (FileStream fs = new FileStream("user.json", FileMode.Create))
{
    Person tom = new Person("Tom", 37);
    JsonSerializer.Serialize<Person>(fs, tom, options);
    Console.WriteLine("Data has been saved to file");
}
```



user.json 1 {"Age": 37}

МЕТОД DESERIALIZE()

Для десериализации кода json в объект C# применяется метод **Deserialize()** и его асинхронный двойник **DeserializeAsync()**, которые имеют различные версии. Некоторые из них:

- **object? Deserialize(string json, Type type, JsonSerializerOptions options):** десериализует строку json в объект типа type и возвращает десериализованный объект. Последний необязательный параметр options позволяет задать дополнительные опции десериализации
- **T? Deserialize<T>(string json, JsonSerializerOptions options):** десериализует строку json в объект типа T и возвращает его. **ValueTask<object?> DeserializeAsync(Stream utf8Json, Type type, JsonSerializerOptions options, CancellationToken token):** десериализует данные из потока utf8Json, который представляет объект JSON, в объект типа type. Последние два параметра необязательны: options позволяет задать дополнительные опции десериализации, а token устанавливает CancellationToken для отмены задачи. Возвращается десериализованный объект, обернутый в ValueTask
- **ValueTask<T?> DeserializeAsync<T>(Stream utf8Json, JsonSerializerOptions options, CancellationToken token):** десериализует данные из потока utf8Json, который представляет объект JSON, в объект типа T. Возвращается десериализованный объект, обернутый в ValueTask

МЕТОД DESERIALIZE()

```
JsonSerializerOptions options = new JsonSerializerOptions()
{
    WriteIndented = true
};
using (FileStream fs = new FileStream("user.json", FileMode.Create))
{
    Person tom = new Person("Tom", 37);
    JsonSerializer.Serialize<Person>(fs, tom, options);
    Console.WriteLine("Data has been saved to file");
}

using (FileStream fs = new FileStream("user.json", FileMode.Open))
{
    Person tom = JsonSerializer.Deserialize<Person>(fs);
    Console.WriteLine(tom.Name);
    Console.WriteLine(tom.Age);
}
```

Консоль отладки Microsoft Visual Studio

Data has been saved to file

Tom

37

- T? Deserialize<T>(Stream str, JsonSerializerOptions options): десериализует строку json в объект типа T и возвращает его.

Ссылка: 0

class Person

{

Ссылка: 2

public string Name { get; }

Ссылка: 2

public int Age { get; set; }

Ссылка: 1

public Person(string name, int age)

{

 Name = name;

 Age = age;

}

}

JSON C LIST<T>

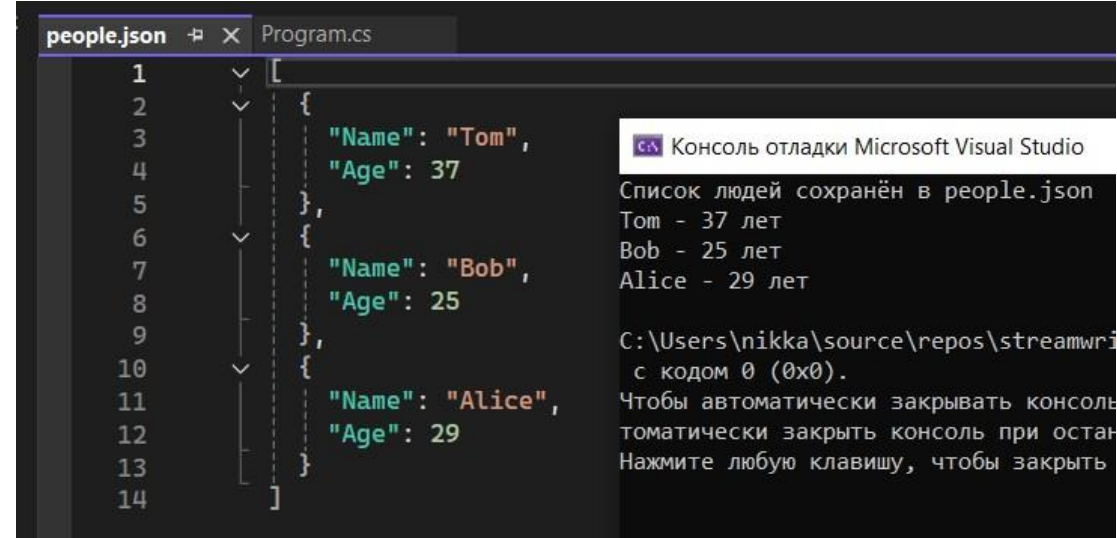
```
static void Main(string[] args)
{
    JsonSerializerOptions options = new JsonSerializerOptions()
    {
        WriteIndented = true
    };

    // Сериализация списка людей
    List<Person> people = new List<Person>()
    {
        new Person("Tom", 37),
        new Person("Bob", 25),
        new Person("Alice", 29)
    };

    using (FileStream fs = new FileStream("people.json", FileMode.Create))
    {
        JsonSerializer.Serialize<List<Person>>(fs, people, options);
        Console.WriteLine("Список людей сохранён в people.json");
    }

    // Десериализация списка
    using (FileStream fs = new FileStream("people.json", FileMode.Open))
    {
        List<Person> loadedPeople = JsonSerializer.Deserialize<List<Person>>(fs);

        foreach (Person p in loadedPeople)
        {
            Console.WriteLine($"{p.Name} - {p.Age} лет");
        }
    }
}
```



The screenshot shows the Visual Studio IDE with two windows open. The 'people.json' window displays the following JSON content:

```
[
  {
    "Name": "Tom",
    "Age": 37
  },
  {
    "Name": "Bob",
    "Age": 25
  },
  {
    "Name": "Alice",
    "Age": 29
  }
]
```

The 'Program.cs' window shows the same C# code as the first block. The 'Консоль отладки Microsoft Visual Studio' (Visual Studio Debug Console) on the right displays the output of the program:

```
Список людей сохранён в people.json
Tom - 37 лет
Bob - 25 лет
Alice - 29 лет
```

Below the output, there is a message: 'C:\Users\nikka\source\repos\streamwr... с кодом 0 (0x0). Чтобы автоматически закрывать консоль, нажмите любую клавишу, чтобы закрыть'.

```
Ссылка: 10
class Person
{
    Ссылка: 2
    public string Name { get; }
    Ссылка: 2
    public int Age { get; set; }

    Ссылка: 3
    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }
}
```


ПРИМЕР ЗАДАНИЯ

Создать класс книга с полями Id, название, автор, количество страниц.

Создать приложение с меню:

1. Создать список и записать его в Json файл.
2. Десериализовать файл в новый список и вывести информацию в консоль.
3. Отсортировать список по названиям.
4. Добавить новую книгу.
5. Сериализовать список в Json файл. И выведите информацию из файла в консоль.

ЗАДАНИЕ С JSON

Цель

Разработать класс для управления коллекцией интернет-мемов с сохранением данных в JSON-файл.

1. Класс Meme

Создать класс с полями:

Id (int) — уникальный идентификатор

Name (string) — название мема

Description (string) — описание мема

CreatedDate (DateTime) — дата создания (автоматически)

SemanticLoad (string) — смысловая нагрузка/суть мема

Ratings (List<int>) — список оценок пользователей (1-10)

2. Класс MemeManagerConsole

Поля: текущий файл(private), текущий список мемов

Конструктор с параметром (path) читает файл и записывает данные в поле список

Разработать методы:

1. Сохранить список в файл
2. Прочитать список из файла и записать его в поле список мемов, так же метод возвращает List<>
3. Записать все меммы в строку, вернуть строку

Следующие методы должны после успешного завершения, вызывать метод «Сохранить список в файл» , затем «Прочитать список из файла», затем вывести строку в консоль используя метод «Записать все меммы в строку»

1. Создать новый мем, взаимодействие с консолью. Редактировать мем, выбор по ID, взаимодействие с консолью
2. Удалить мем из списка по ID, взаимодействие с консолью
3. Добавить оценку к мему, выбор по Id , взаимодействие с консолью

Требования:

1. Проверка введённых данных, если они не верны, изменения не сохраняются
2. Использовать *поле const options*
3. *ID не должны повторяться*